TC789 – TEKNOLOGI BLOCKCHAIN

Chapter 10

Ethereum 101

Architecture, Accounts, Transactions & the EVM

Teguh Indra Bayu, Ph.D.

FTI – Universitas Kristen Satya Wacana (UKSW)

Source

Mastering Blockchain
2nd Edition
Imran Bashir
Packt Publishing

Pages 417–459

Undergraduate Introductory Blockchain Lecture

Learning Objectives

What you will be able to explain after this lecture

01 Explain why Ethereum was created and how it differs from Bitcoin

02 Describe Ethereum's state machine model and the role of the World State

03 Distinguish between Externally Owned Accounts (EOAs) and Contract Accounts (CAs)

04 Define and explain the structure of Ethereum transactions and messages

05 Explain how the Ethereum Virtual Machine (EVM) executes smart contract code

06 Describe the role of Gas in controlling computation on Ethereum

07 List Ethereum's network types (Mainnet, Testnet, Private Net) and their purposes

08 Identify the 8 native precompiled contracts in Ethereum Byzantium

Chapter Overview

Mastering Blockchain Ch. 10 — Topic Map

01

Introduction & History

Origins, Yellow Paper, state machine model

02

Bird's Eye View

Sending Ether — a complete walkthrough

03

Network Types

Mainnet, Testnet (Ropsten), Private Net

04

Ecosystem Components

Keys, Accounts, Transactions, EVM, Smart Contracts

05

The EVM

Stack machine, Gas, execution environment

06

State Storage

World State, Tries, Transaction Receipts

Introduction: Origins of Ethereum

Why was Ethereum created?

Vitalik Buterin conceptualized Ethereum in November 2013.

Core innovation: A Turing-complete scripting language for blockchain — enabling arbitrary smart contracts.

Bitcoin limitation: Bitcoin's scripting is limited; only basic conditional logic is supported.

Goal: Enable decentralized applications (DApps) on a programmable blockchain.

Formal spec: The Yellow Paper by Dr. Gavin Wood — anyone can implement a compliant Ethereum client.

Ethereum Release History

Version	Date	Notes
Olympic	May 2015	First public testnet
Frontier	Jul 2015	Live launch
Homestead	Mar 2016	Stability improvements
Byzantium	Oct 2017	Metropolis Phase 1 ★
Constantinople	Planned ~2018	Metropolis Phase 2
Serenity	Future	Final — PoS based

★ Current version covered in this chapter

Ethereum: A Transaction-Based State Machine

The fundamental model of Ethereum

- Ethereum is modeled as a transaction-based state machine
- A genesis state transforms into a final state by executing transactions incrementally
- The Ethereum State Transition Function (Y) drives every state change
- Mining plays the central role in state transitions
- The World State is the global state — a mapping from all addresses to account states
- Each state is universally agreed upon by the network (consensus)

```
Y(σ, T) = σ'    (World State × Transaction → New State)
```

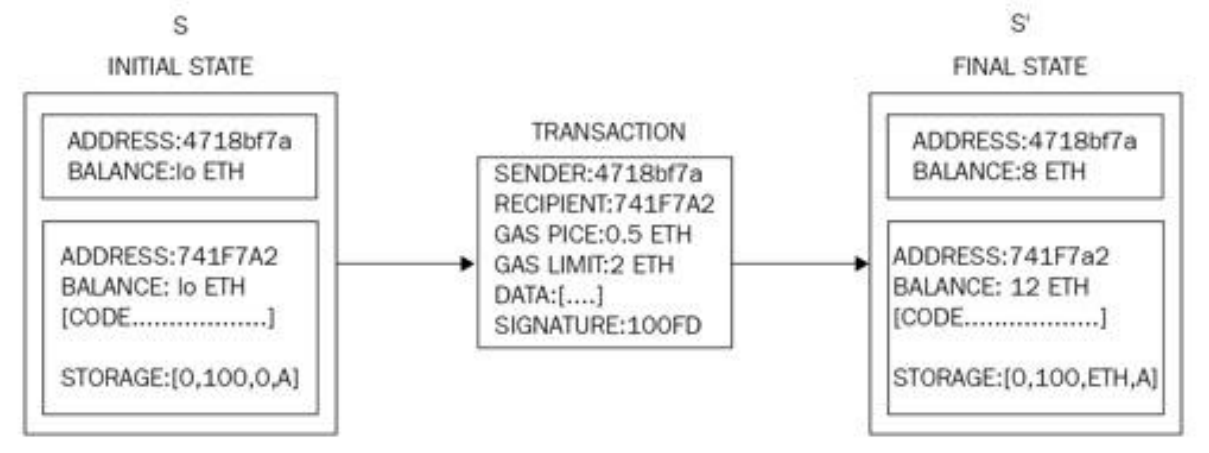


Figure: Ethereum State Transition Function (p.422)



Figure: Etherscan block explorer — tracking a transaction (p.426)

Bird's Eye View: Sending Ether

Step-by-step — from wallet to blockchain

1. Receiver shares Ethereum address (or QR code) with sender
2. Sender enters amount + recipient address in wallet software (e.g., Jaxx)
3. Transaction is digitally signed by sender and broadcast to the P2P network
4. Miners pick up the transaction for verification (unconfirmed state)
5. Miner runs Proof-of-Work (PoW) — finds valid nonce by repeated hashing
6. Block is broadcast, verified by network, and added to the blockchain
7. Sender's balance decreases; receiver's wallet shows the received Ether



Jaxx wallet — sending Ether (p.423)



Received Ether in wallet (p.425)

Ethereum Network Types

Three types of networks for different purposes

Mainnet

Chain ID: 1

The live, production Ethereum network. Current version: Byzantium (Metropolis). Explorer: etherscan.io. Real ETH is used here.

Use: Production smart contracts & DApps

Testnet (Ropsten)

Chain ID: 3

Primary test network. Used for testing smart contracts and DApps before mainnet deployment. No real value — free test ETH. Includes Kovan (ID:42) and Rinkeby (ID:4) as special testnets.

Use: Development & testing — no real ETH

Private Net

Custom Chain ID

A new private blockchain created by generating a new genesis block. Used for enterprise/private permissioned blockchains. A private group of entities can run their own chain.

Use: Permissioned / enterprise blockchains

Ethereum Ecosystem Components

The building blocks of the Ethereum stack

Keys & Addresses

Private key → Public key → 20-byte address (Keccak hash)

Accounts (EOA & CA)

Two types: user-controlled (EOA) and code-controlled (CA)

Transactions & Messages

Signed data packets that drive state transitions

Ether (ETH)

Native currency; used to pay for gas (computation)

The EVM

Stack-based virtual machine executing contract bytecode

Smart Contracts

Turing-complete programs running on the EVM

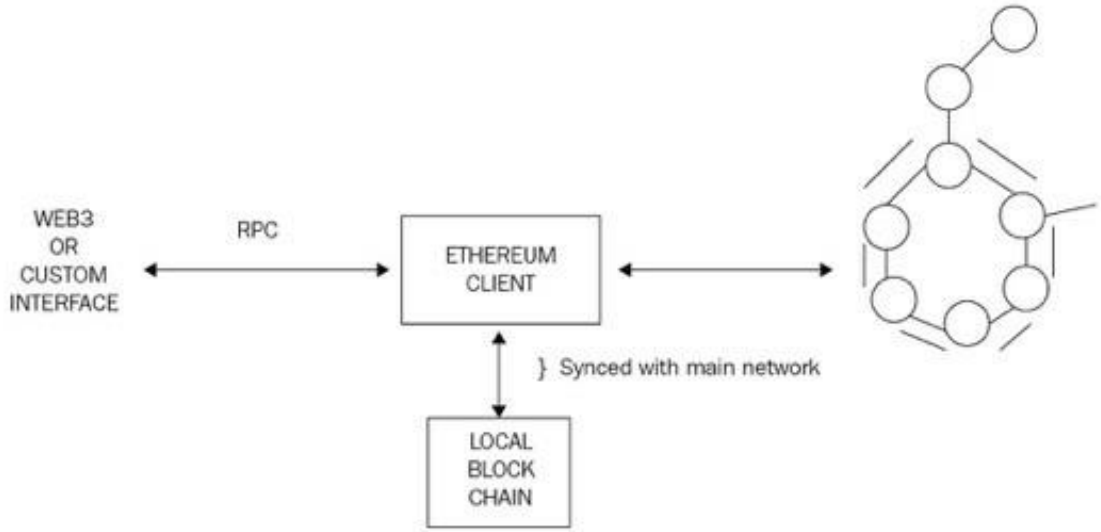


Figure: Ethereum stack — P2P network, Geth client, web3.js (p.431)

The Ethereum Stack:

1. Ethereum P2P Network (foundation)
2. Ethereum Client — Geth (node software)
3. web3.js library (DApp interface via RPC)

Keys, Addresses & Accounts

Identity and ownership on Ethereum

Keys & Addresses

Private Key: 256-bit random integer (secp256k1 curve)

Public Key: derived from private key via ECDSA

Address: rightmost 160 bits of Keccak(Public Key)

Addresses are 20 bytes (40 hex characters)

Example (from chapter):

```
Private key:   b51928c2...954ffc4
Public key:    3aa5b8ee...
Address:       0x77b4b569...6f32
```

EOA	CA
Externally Owned Account	Contract Account
Ether balance	Ether balance
Controlled by private key	Triggered by transactions/messages
No associated code	Has associated EVM code
Associated with a human user	Not tied to any user
Key-value store	Key-value store
Can send transactions	Can call other contracts

Account State

What the Ethereum blockchain stores per account

Nonce	Counts transactions sent from address (EOA) or contracts created (CA). Used for replay protection.
Balance	Amount of Wei held by the account. (1 ETH = 10^{18} Wei)
Storage Root	Root of the Merkle Patricia Trie encoding the account's storage contents.
Code Hash	Keccak-256 hash of the account's smart contract code. Immutable after creation. For EOAs, this is the hash of an empty string.

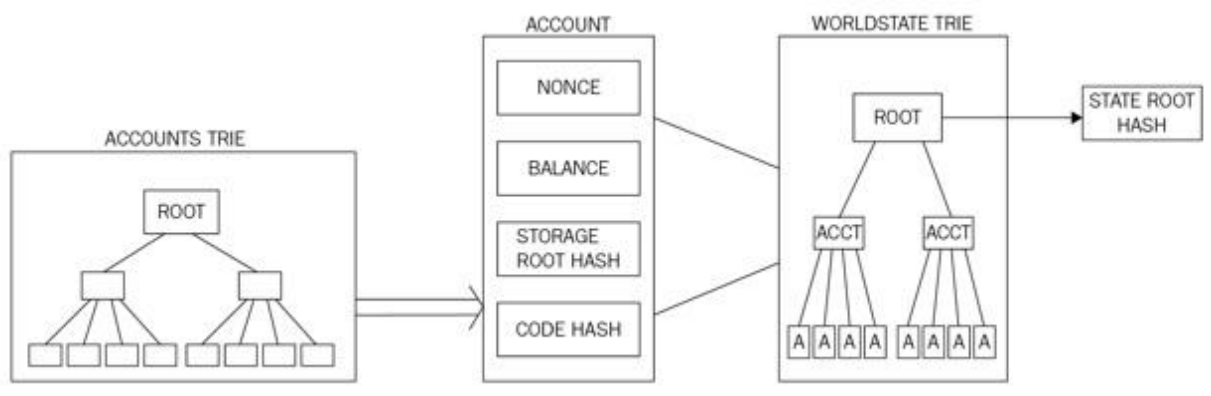


Figure: World state trie, account tuple, and state root hash relationship (p.446)

World State & Storage

World State: global mapping of address → account state

Serialized using RLP (Recursive Length Prefix) encoding

Stored as a Merkle Patricia Trie for efficient verification

State root hash is embedded in each block header

Transaction and receipt tries are also stored per block

Transactions: Structure & Types

Digitally signed data packets that drive state change

Message Call Transaction

Passes messages between accounts. Produces a message call. No new account is created.

Contract Creation Transaction

Creates a new Contract Account with associated EVM code. Runs init code once, then deploys body code.

Transaction Fields	Description
Nonce	Unique counter — prevents replay attacks
Gas Price	Wei per unit of gas willing to pay
Gas Limit	Maximum gas units allowed for the transaction
To	20-byte recipient address
Value	Amount of Wei to transfer
Signature (V,R,S)	ECDSA digital signature — proves ownership
Init / Data	Init code (contract creation) or input data (message call)

ECDSASIGN(Msg, PrivKey) = (V, R, S) — V: recovery ID — R: x-coord on elliptic curve point — S: signature scalar

Transaction Lifecycle & Validation

From wallet to confirmed block

1	2	3	4	5	6
Sign & Broadcast	**Transaction Pool**	**Miner Picks Up**	**Proof of Work**	**Block Broadcast**	**Confirmed**
Wallet signs with private key, sends to P2P network	Waits for miner verification (unconfirmed)	Highest-paying transactions selected first	Miner hashes block + nonce until target met (Ethash)	Valid block propagated to all network nodes	Block added to chain; state updated; rewards paid

Transaction Validation Requirements (all 5 conditions must be met)

- Well-formed RLP encoding — no trailing bytes
- Valid digital signature (ECDSA)
- Nonce must equal sender's current account nonce (replay protection)
- Gas limit ≥ intrinsic gas cost of the transaction
- Sender account must have sufficient Ether balance to cover gas + value

Ref: Transaction → Transaction Trie → Block Header — see textbook p.436

Messages, Calls & Transaction Substate

Internal communications and execution side-effects

Transactions

Created by external actors (EOAs)

Digitally signed and broadcast

Stored on the blockchain

Drive state transitions

Messages

Generated by contract code

Via CALL / DELEGATECALL opcodes

Never stored on-chain

Internal — not broadcast

Calls

Local read-only invocations

Run only on local node VM

No state change, no gas used

Available via web3.js API

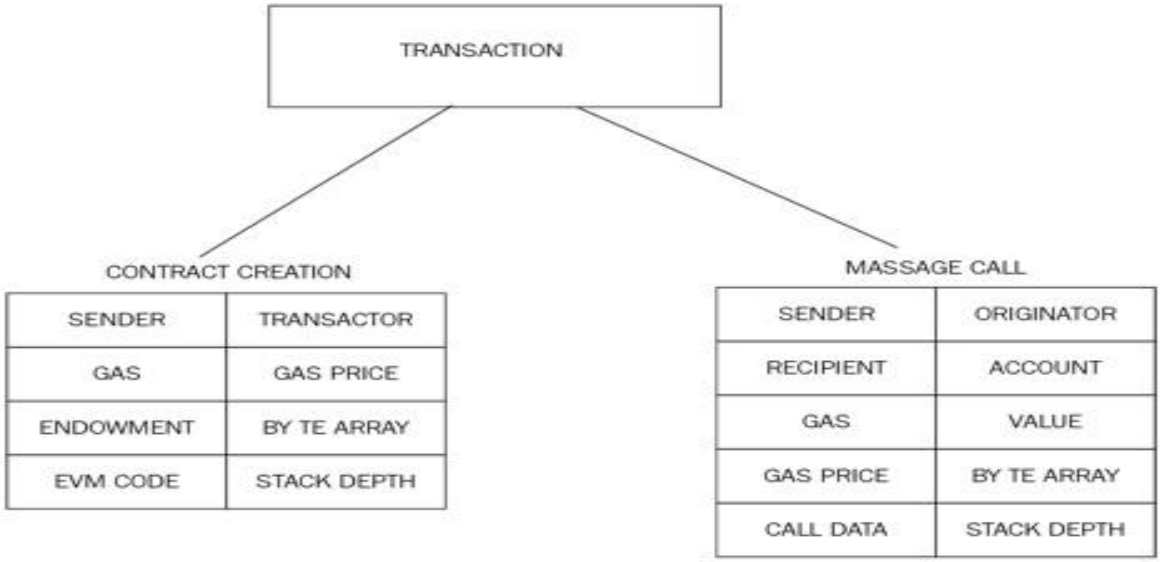


Figure: Transaction types and required parameters (p.439)

Transaction Substate (created after execution)

Suicide set — accounts disposed of after execution

Log series — event-triggered checkpoints (for frontends)

Refund balance — partial gas cost offset

Touched accounts — empty ones deleted at end

Ether & Gas

Currency and computation metering in Ethereum

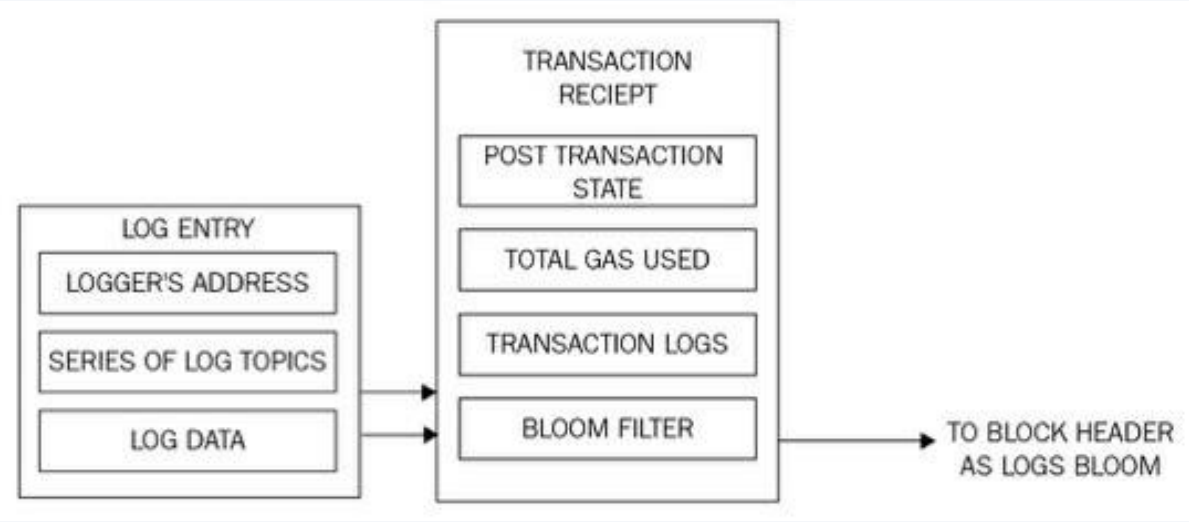


Figure: Transaction receipts and logs bloom (p.447)

Ether (ETH) — native currency of Ethereum

Minted by miners as block rewards

Used to pay for gas (computation on EVM)

ETH (Ethereum, forked) vs ETC (Ethereum Classic, original)

DAO hack (2016) led to the ETH/ETC hard fork split

Ether Denominations

Unit	Alt. Name	Wei Value
Wei	Wei	1
KWei	Babbage	1,000
MWei	Lovelace	1,000,000
GWei	Shannon	1,000,000,000
Micro Ether	Szabo	10^{12}
Milli Ether	Finney	10^{15}
Ether	Ether	10^{18} **(1,000,000,000,000,000,000)**

Gas: fee paid per unit of computation. Prevents infinite loops and network abuse.

The Ethereum Virtual Machine (EVM)

The engine that executes smart contract code

Architecture:	Stack-based (LIFO), 256-bit word size
Stack depth:	Maximum 1024 elements
Turing-complete:	Yes — but limited by Gas requirement
Isolation:	Sandboxed — no network/filesystem access
Memory:	Volatile byte array (like RAM, cleared after execution)
Storage:	Non-volatile key-value store (permanently on-chain)
Endianness:	Big-endian by design
Exception handling:	Halts on out-of-gas, invalid opcode, stack overflow

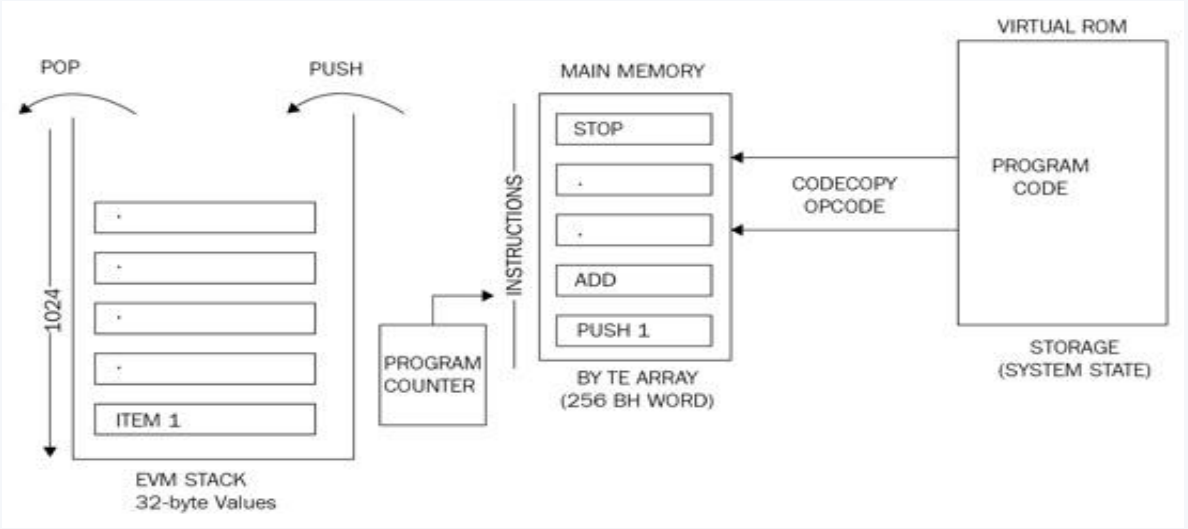


Figure: EVM operation — virtual ROM → main memory → stack (p.450)

Why Gas?

Every EVM instruction has a fixed gas cost

Gas limits execution → prevents infinite loops (DoS attacks)

If gas runs out mid-execution: state reverts (except fees paid)

Gas price (in Wei) × gas used = total Ether fee paid to miners

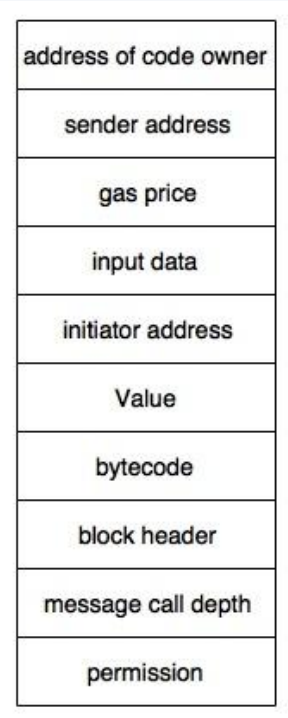
Future: eWASM (WebAssembly backend) to improve EVM performance

EVM: Execution Environment & Machine State

What the EVM needs to run — and what it tracks

Execution Environment (10-element tuple)

1. Current system state (σ)
2. Remaining gas for execution
3. Address of code-owning account
4. Sender's address
5. Originating transaction address
6. Gas price
7. Input data / transaction data (byte array)
8. Account that initiated execution
9. Value in Wei
10. Block header of current block



Execution environment tuple (p.452)



Machine state tuple (p.455)

Machine State (internal EVM state)

Available gas remaining

Program Counter (PC)

Memory contents (byte array)

Active memory words count

Stack contents

EVM Halt Conditions

Out of gas (OOG exception)

Invalid instruction/opcode

Stack underflow or overflow (>1024)

Invalid JUMP destination

STOP / RETURN / SUICIDE opcodes

Smart Contracts & Native Precompiled Contracts

Programmable logic and built-in cryptographic functions

Smart Contracts: Turing-complete programs that run on the EVM. Written in Solidity (or Vyper), compiled to EVM bytecode. Triggered by transactions or messages. Two categories: user-programmed contracts and precompiled native contracts.

Address	Function Name	Description	Gas
0x1	ECDSARECOVER	Recover EC public key from ECDSA signature (V,R,S)	3,000
0x2	SHA-256	SHA-256-bit hash of input data	Input-size based
0x3	RIPEMD-160	RIPEMD 160-bit hash — outputs 20-byte value	Input-size based
0x4	Identity (DataCopy)	Output = Input; used for data copying	15 + 3×[Id/32]
0x5	BigModExp	Big integer modular exponentiation — enables RSA verification	Variable
0x6	EC Point Addition	Elliptic curve point addition (enables ZK-SNARKs)	150
0x7	EC Scalar Multiply	Elliptic curve scalar multiplication (enables ZK-SNARKs)	6,000

Note: These native contracts run within the Ethereum client (Geth/Parity) as compiled code — not user-written Solidity contracts.

Key Terms & Definitions

Essential vocabulary for Ethereum 101

Ethereum	Decentralized, Turing-complete blockchain platform for smart contracts and DApps
EVM	Ethereum Virtual Machine — sandboxed, stack-based runtime executing contract bytecode
Gas	Unit of computation; every instruction costs gas; limits execution and prevents abuse
Wei	Smallest denomination of Ether: 1 ETH = 10^{18} Wei
EOA	Externally Owned Account — private key controlled, associated with humans
CA	Contract Account — code-controlled, holds contract logic and state
Nonce	Counter incremented per transaction; prevents replay attacks
World State	Global mapping of all Ethereum addresses to account states
RLP	Recursive Length Prefix — Ethereum's binary serialization format for storage/transmission
Yellow Paper	Formal mathematical specification of the Ethereum protocol by Dr. Gavin Wood

Benefits vs Limitations

A balanced view of Ethereum's strengths and challenges

✓ Benefits

- Turing-complete — enables arbitrary complex smart contracts
- Gas mechanism protects network from infinite loops and DoS attacks
- Sandboxed EVM ensures deterministic, secure code execution
- Flexible account model: EOAs for users, CAs for contracts
- Rich ecosystem: Testnet, block explorers, wallet software
- Open protocol — anyone can implement a compliant client
- 8 precompiled native contracts for efficient cryptographic operations

✗ Limitations

- Proof-of-Work (Byzantium) is energy-intensive; Serenity targets PoS
- EVM is slower than native code (eWASM research addresses this)
- Gas complexity adds friction for developers and end users
- DAO hack (2016) required a controversial hard fork → ETH / ETC split
- Scalability: low throughput compared to centralized systems
- Smart contract bugs are irreversible once deployed (immutability risk)
- Steep learning curve for new developers (cryptography, EVM, Solidity)

Ethereum vs Bitcoin

Key differences between the two blockchain platforms

Aspect	Bitcoin	Ethereum
Creator	Satoshi Nakamoto (2009)	**Vitalik Buterin (2013/2015)**
Primary Goal	Peer-to-peer digital cash	**Programmable decentralized platform**
Scripting	Limited (non-Turing-complete)	**Turing-complete (EVM bytecode)**
Smart Contracts	Basic (UTXO conditions only)	**Full smart contract support**
Account Model	UTXO (Unspent Transaction Output)	**Account-based (EOA / CA)**
Consensus (Byzantium)	Proof of Work (SHA-256)	**Proof of Work (Ethash, ASIC-resistant)**
Block Time	~10 minutes	**~15 seconds**
Computation Fee	Transaction fee (satoshis/byte)	**Gas (Wei per computation unit)**
State	UTXO set only	**Full World State (account states)**

Source: Mastering Blockchain, 2nd Ed., Ch. 10 (pp. 417–459)

State Storage: World State & Transaction Receipts

How Ethereum records and verifies its state

World State (σ)

Global mapping: address → account state

Serialized as Merkle Patricia Trie

RLP-encoded for storage/transmission

State root hash stored in block header

Transaction Trie

All transactions in a block indexed as a trie

Keccak root hash stored in block header

Enables efficient proof of inclusion

Transaction Receipts Trie

Post-execution state for each transaction

Contains: gas used, log entries, bloom filter

Byzantium adds success/failure flag (1/0)

Receipt root hash in block header

Bloom Filter: Compact 256-byte structure derived from log entries — embedded in block header as logsBloom for efficient log searching

Putting It All Together

The complete Ethereum transaction flow

User / EOA
Signs transaction with private key

P2P Network
Transaction broadcast

Miner Node
Picks up, validates

EVM Executes
Runs bytecode, consumes gas

World State
Updated via state transition

Block Added
Miner rewarded with ETH

Accounts & State

EOA: private key controlled

CA: code controlled

Account state: nonce, balance, storage root, code hash

World State: Merkle Patricia Trie

Transactions & EVM

Transactions: message call or contract creation

EVM: stack-based, 256-bit, sandboxed

Gas limits computation

Messages: internal contract-to-contract

Network & Contracts

Mainnet (ID:1), Ropsten Testnet (ID:3)

Private nets for permissioned use

8 precompiled native contracts

Smart contracts: Turing-complete on EVM

Key Takeaways

1. Ethereum extends Bitcoin with Turing-complete smart contract capability (Vitalik Buterin, 2013)
2. Ethereum is a transaction-driven state machine — the World State maps all addresses to account states
3. Two account types: EOA (private key / human) and CA (code / contract)
4. Gas meters every EVM computation — it prevents infinite loops and charges for computation
5. The EVM is sandboxed, stack-based, and 256-bit — it runs compiled bytecode from smart contracts
6. Transactions are signed data packets; messages are internal contract-to-contract communications
7. State is stored in Merkle Patricia Tries — efficient, cryptographically verifiable
8. The Yellow Paper is the formal Ethereum specification — open for anyone to implement
9. 8 native precompiled contracts provide efficient cryptographic primitives on-chain
10. Active research on eWASM, ZK-SNARKs, and PoS aims to improve speed, privacy, and scalability

Discussion & Q&A

Applying the concepts

Q1 Why is Ethereum described as a "state machine"? What is the World State?

Q2 What is the key difference between an EOA and a Contract Account?

Q3 Explain why Gas is necessary in the Ethereum ecosystem.

Q4 What happens when an EVM transaction runs out of gas mid-execution?

Q5 Why did the Ethereum community hard fork in 2016, and what resulted from it?

Q6 A developer wants to use ZK-SNARKs in their DApp. Which precompiled contracts are relevant?

Reference: Mastering Blockchain 2nd Ed. (Imran Bashir, Packt) — Chapter 10: Ethereum 101, pp. 417–459